

PROGRAMMING II

OBJECT-ORIENTATION AND UML

Michele Pasqua, PhD



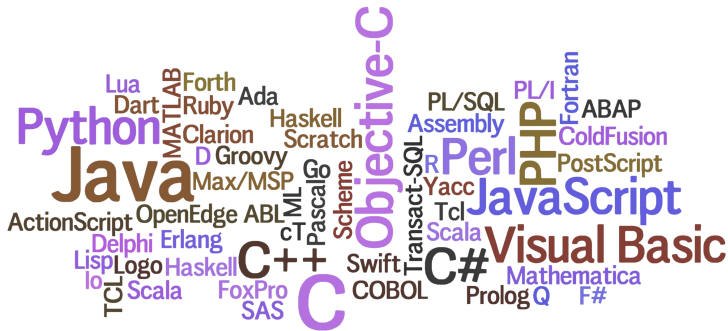
UNIVERSITÀ
di **VERONA**
Dipartimento
di **INFORMATICA**

A.Y. 2025/2026

PROGRAMMING LANGUAGES



PROGRAMMING LANGUAGES ZOO



There is no universal programming language (even if someone tried in late '70)

PROGRAMMING LANGUAGES ZOO (CONT'D)

PYTHON



JAVA



C++



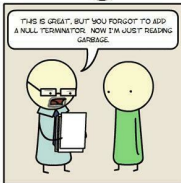
UNIX SHELL



ASSEMBLY



C



LATEX



HTML



WHY DO WE NEED DIFFERENT PROGRAMMING LANGUAGES?

"Computer Science is no more about computers than Astronomy is about telescopes."
E.W. Dijkstra

If all you have is a hammer, everything looks like a nail



WHY DO WE NEED DIFFERENT PROGRAMMING LANGUAGES? (CONT'D)

Modern programmers need **abstraction**

Back in '50/'60 the goal was machine code efficiency

- ▶ Direct connection between the language and the **hardware implementation**
- ▶ Programmers cheap, machine expensive: *Keep the machine busy!*

Nowadays the goal is program code efficiency

- ▶ Direct connection between the language and the **software design**
- ▶ Programmers expensive, machine cheap: *Keep the programmer busy!*

PROGRAMMING PARADIGMS

Imperative/Procedural languages

- ▶ Based on the notion of state (memory locations containing values)
- ▶ Instructions are commands changing the state (e.g., assignments)
- ▶ Code organized in logical blocks for reuse
- ▶ C, Pascal, FORTRAN, COBOL, ...

Declarative languages

- ▶ Based on the notion of function or relation
- ▶ Instructions are declarations of new values
- ▶ Lisp, Haskell, Prolog, SQL, ...

PROGRAMMING PARADIGMS (CONT'D)

Functional languages (declarative)

- ▶ The result of a program is the explicit value of an expression
- ▶ Function definition and application, recursion
- ▶ Programming boils down to define a function computing a result

```
// Haskell 1
add :: Integer -> Integer -> Integer // function declaration 2
add x y = x + y // function definition 3
4
main = do 5
  putStrLn "The addition of the two numbers is:" 6
  print(add 2 5) // function call 7
```


PROGRAMMING PARADIGMS (CONT'D)

Logic languages (declarative)

- ▶ Program result is the value of some variables satisfying some relations
- ▶ Logical implications between formulas, rewriting rules
- ▶ Programming boils down to define the relations between variables

```
// Prolog                                     1
happy(sam).                                   // fact 1                             2
listens2Music(mia).                          // fact 2                             3
listens2Music(sam):- happy(sam).              // rule 1                             4
playsAirGuitar(mia):- listens2Music(mia).     // rule 2                             5
playsAirGuitar(sam):- listens2Music(sam).     // rule 3                             6
```

Query: ?- playsAirGuitar(mia), playsAirGuitar(sam).

// Answer: yes

PROGRAMMING PARADIGMS (CONT'D)

Object-Oriented languages

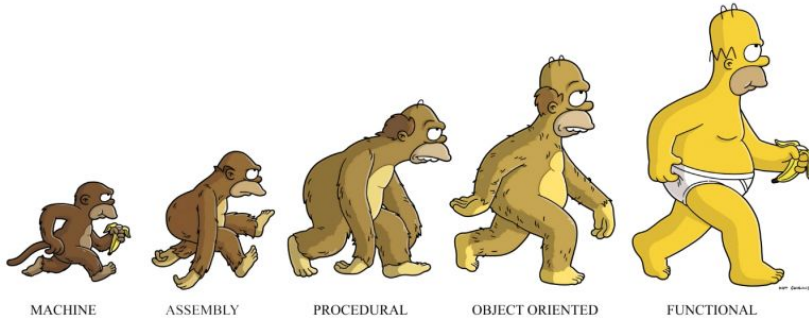
- ▶ Imperative language with some injection of declarative concepts
- ▶ Objects contain data (imperative trait)
- ▶ Objects contain methods that operate on the data (declarative trait)
- ▶ Based on message-passing
- ▶ Abstractions: encapsulation, inheritance, ...
- ▶ Smalltalk, Java, C++

Trend of developing **multi-paradigm** languages (not always a good thing)

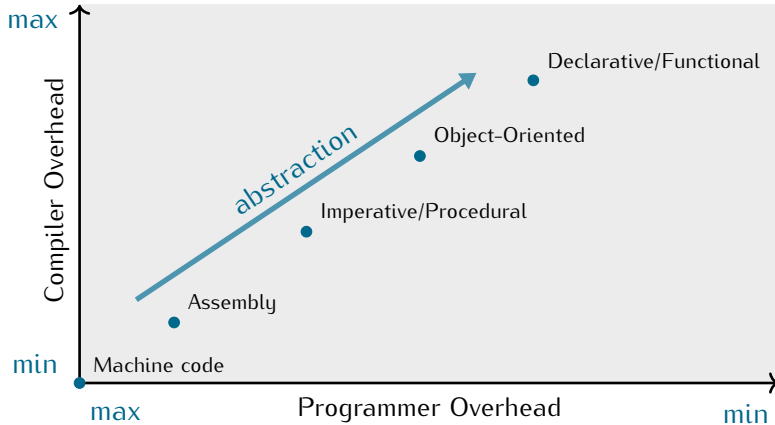
A BRIEF OF HISTORY



TOWARDS ABSTRACTION



TOWARDS ABSTRACTION (CONT'D)



OBJECT-ORIENTATION



PROCEDURAL LANGUAGES

- ▶ Group instructions into code blocks
- ▶ Data: vect[20]
- ▶ Functions: sort()
- ▶ Implicit read and write relations between data and functions

```
// C program to handle integer vectors

int vect[20];

void sort() { /* sort */ }
int search(int n){ /* search */ }
void init() { /* init */ }

int i;

void main() {
    init();
    sort();
    search(13);
}
```

PROCEDURAL LANGUAGES

- ▶ No explicit relation between vectors and operations on vectors
- ▶ No vector boundaries check
 - ▶ `for (i=0; i <= 21; i++) vect[i]=0;`
- ▶ No vector initialization check
 - ▶ Is `init()` executed?

```
// C program to handle integer vectors

int vect[20];

void sort() { /* sort */ }
int search(int n){ /* search */ }
void init() { /* init */ }

int i;

void main() {
    init();
    sort();
    search(13);
}
```

In general, **vector** cannot be a standalone, modular concept!

PROCEDURAL LANGUAGES LIMITATIONS

The **programmer** is responsible for the integrity and confidentiality of data

- ▶ No constraints on read and write relations between data
- ▶ External functions can influence vector data

On the long run

- ▶ Every function can read and write every data (bad)
- ▶ Too many implicit relations to **manually** check
- ▶ Code becomes difficult to understand and maintain (**spaghetti code**)

OBJECTS TO THE RESCUE

Procedural languages

- ▶ Program defines data and then calls subprograms acting on data

Object-Oriented languages

- ▶ Program creates objects that **encapsulate** both data and operations on data

Object-Orientation is simply a new way of organizing programs

- ▶ Cannot do anything that cannot be done using a procedural language

OBJECT-ORIENTED LANGUAGES ADVANTAGES

Object-Oriented languages

- ▶ Allow programmers to use code blocks without knowing all the picture
- ▶ Make code reuse a real possibility
- ▶ Simplify maintenance and evolution

These are critical points when

- ▶ Considering a large codebase
- ▶ Many people and teams work together

OBJECT-ORIENTED LANGUAGES ADVANTAGES (CONT'D)

Programs with less than 30 LoC

- ▶ Spaghetti code is understandable and faster to write
- ▶ Spaghetti code is reasonably easy to maintain

Programs with more than 1000 LoC

- ▶ Spaghetti code is incomprehensible
- ▶ Spaghetti code probably does not work
- ▶ Spaghetti code is very hard to maintain

Only programs with more than 1000 LoC benefit from Object-Orientation really

OBJECT-ORIENTED DESIGN

Given a system, with components and relations among them

- ▶ Identify the components
- ▶ Define components interface
- ▶ Define how components interact with each other through their interface
- ▶ Minimize relations among components

Programmers are forced into this mindset by the language

OBJECT-ORIENTED DESIGN (CONT'D)

Defines a new component **type**

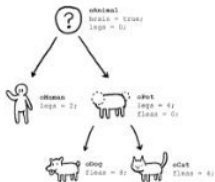
- ▶ Object (and class)
- ▶ Both data and operations on data are within the same module
- ▶ Allows defining a more precise interface

Defines a new kind of **relation**

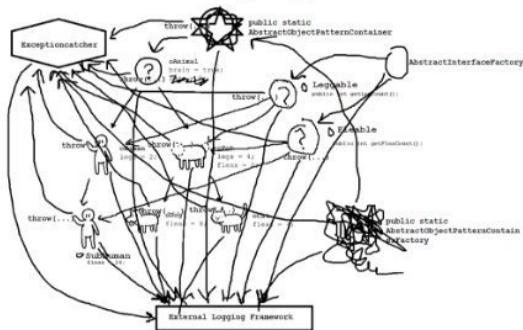
- ▶ Message passing between objects
- ▶ Read and write operations are limited to the object scope

OBJECT-ORIENTED DESIGN (CONT'D)

What OOP users claim



What actually happens



UNIFIED MODELING LANGUAGE



UML



Unified Modeling Language

- ▶ Standard modeling and specification language, widely adopted
- ▶ To **graphically** define and document an Object-Oriented system

CLASS DIAGRAM

UML supports many kind of diagrams (activity, use case, sequence)

We are interested in **Class Diagrams**

- ▶ Capture abstract concepts *// e.g., cars*
- ▶ Model data associated to the concepts *// e.g., car manufacturer*
- ▶ Model relations between concepts *// e.g., cars and trucks are vehicles*
- ▶ Model the behavior of the concept *// e.g., cars cannot fly*

CLASS

A **class** represents a set of objects (e.g., a type)

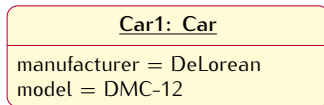
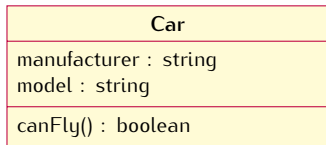
- ▶ Having common properties
- ▶ Having autonomous existence
- ▶ An instance of a class is an **object** of the type that the class represents

Car
manufacturer : string model : string
canFly() : boolean

OBJECT

An **object** represents a physical or logical item

- ▶ Has identity and attributes
- ▶ Has operation it can perform (behavior)
- ▶ Has messages it can send and receive



CLASSES AND OBJECTS

A class

- ▶ Is the description of object structure
- ▶ Defines data, called **attributes** or **fields**
- ▶ Defines operations on data, called **methods**
- ▶ Defines object creation procedures, called **constructors**

An object

- ▶ Is an instance of a class
- ▶ Contains actual data values (state)
- ▶ Has an identity, to distinguish objects from the same class

CLASSES AND OBJECTS (CONT'D)

A class is a type definition

- ▶ Typically no memory is allocated until an object is created from the class

The creation of an object is called **instantiation**

- ▶ The created object is often called an **instance** of the class

There is no limit to the number of objects that can be created from a class

- ▶ ...heap permitting

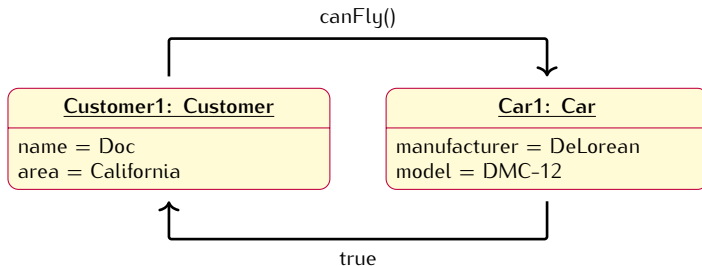
Each object is independent

- ▶ Interacting with one object does not affect the others

How OBJECTS INTERACT

Objects communicate by **message passing**

- ▶ Not by procedure call or direct access to objects local data
- ▶ A message is a service request/response



OBJECT INTERFACE

An object **interface** is the set of messages it can receive

- ▶ Any other message is illegal
- ▶ The message is mapped to a function within the object
- ▶ Simply the subsets of methods that other objects can call

Car
<i>public</i> manufacturer : string <i>private</i> model : string
<i>public</i> canFly() : boolean <i>private</i> setFrameNumber(uint num)

Field and method visibility:

public means accessible from everywhere

private means accessible within the class only

ENCAPSULATION AND INFORMATION HIDING

Encapsulation: bundling data and operations on data within a single unit

Information hiding: restricting external access to certain parts of the unit

Benefits

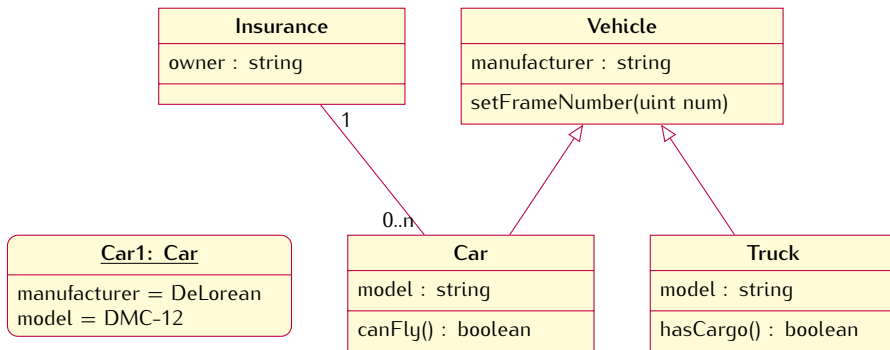
- ▶ To use an object, it is only needed to understand its interface
- ▶ Object implementation does not need external code
- ▶ Object implementation can change without the need of other code refactoring
- ▶ Code changes occur in a single location (less errors)

CLASS ASSOCIATIONS

Logical relations between classes

// we will see more later in the course

► Inheritance



OBJECT-ORIENTATION PILLARS

Encapsulation



Polymorphism



Inheritance



Abstraction





Q + A

